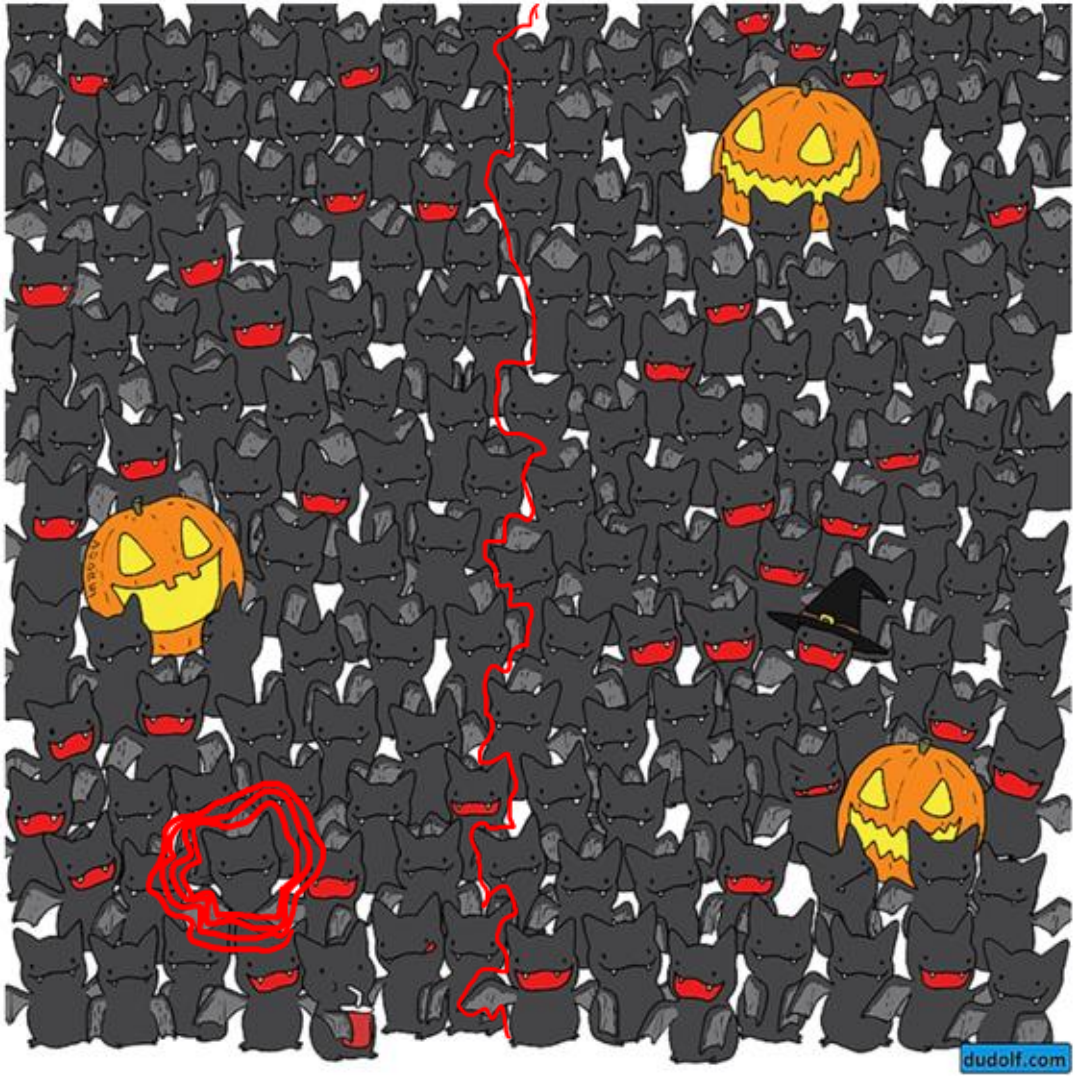


Find the cat

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

Exam 2 Review

Updates

1. Exam 2 Thursday
   a. Exam review out
   b. Practice exam out
   c. 2nd chance next week Thursday

study guide

no class Thursday

2. No HW due this week

3. MP5 Wallet due next week Tuesday

CS 340 - Computer Systems

LG: Understanding, at a high level, how we get from electricity to a website.

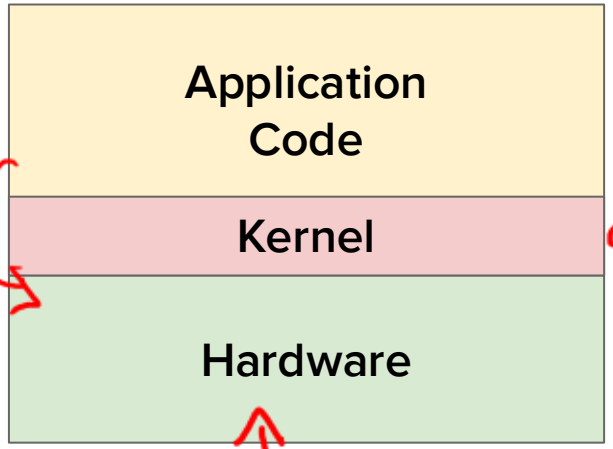
LG: Developing mental models of how different computer systems work so that you can understand more complex ideas if needed later.

LG: Getting experience working on unstructured, substantial coding projects.

Agenda

- Memory in a program
- Processes and threads
- Bits to represent values
- Synchronization
- Exam 2 practice exam review

Big Ideas



has the illusion of virtual memory

- part of the OS
- code
- special permission to connect hardware

To get into Kernel mode
- interrupt
- exception
- system call

made up of

transistors → gate → bigger logic

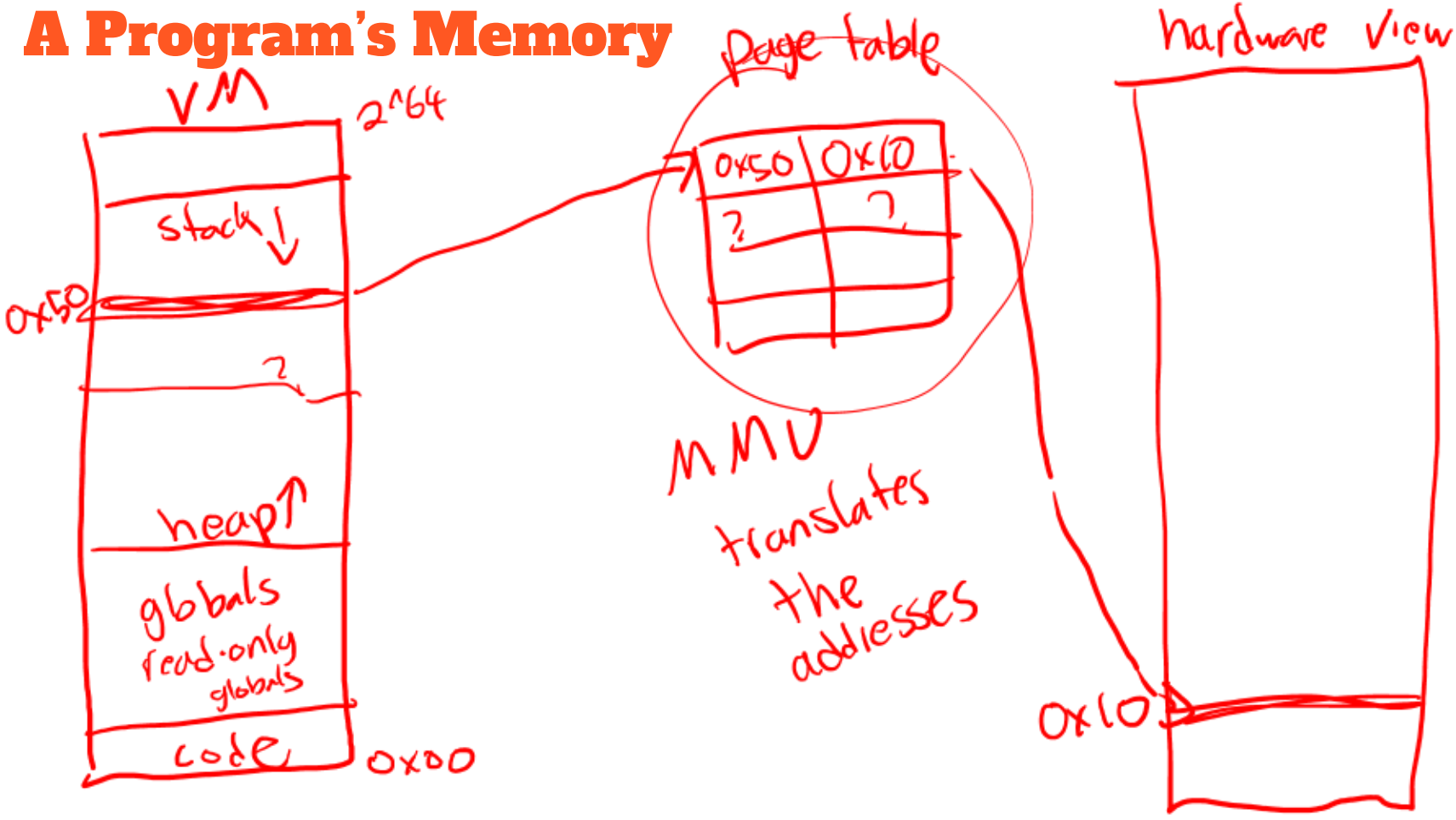
storage for Is and Os

CPU ← ALU

Memory

Disk

A Program's Memory



Find someone nearby

A) What is your name

B) Favorite thing you ate this week

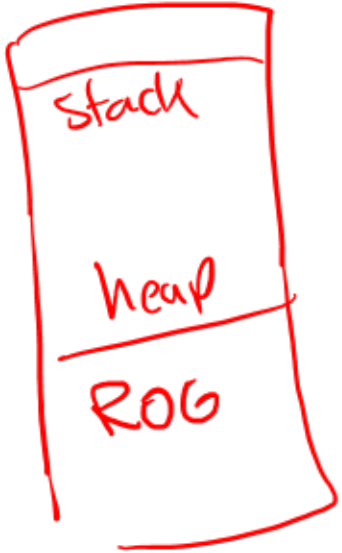
What memory segment is there an error in?

clicker.cs.illinois.edu

Q2

~Code~
340

```
11  int main() {
12      int arr[5] = {1, 2, 3, 4, 5};
13      printf("%i\n", arr[5]);
14  }
```



heap? no
ROG? yes
stack? yes and error

What memory segment is there an error in?

```
13  int main() {
14      int* arr = malloc(4*5);
15  }
```

free(arr)

memory leak :(

clicker.cs.illinois.edu

Q3

~Code~
340

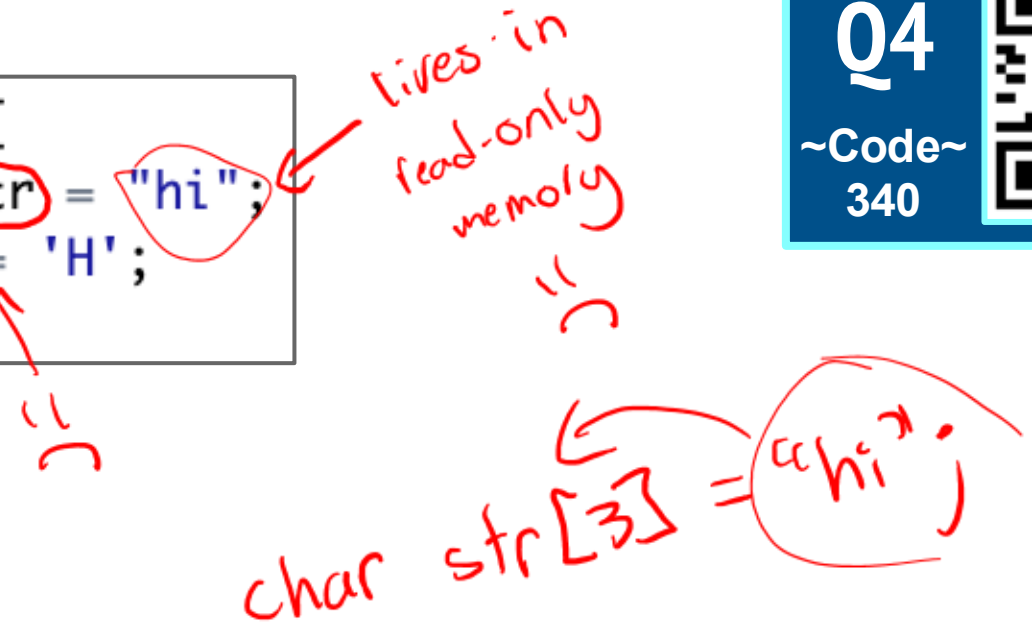
What memory segment is there an error in?

clicker.cs.illinois.edu

Q4

~Code~
340

```
13  int main() {
14      char *str = "hi";
15      str[0] = 'H';
16  }
```



Which are true?

True

clicker.cs.illinois.edu

Q5

~Code~
340

A) To run a program that prints “hello world” bits that represent ‘h’ or 68 need to be stored somewhere physically in memory.

B) To compile and run a program that prints “hello world” to your screen the OS has to manage multiple programs and/or devices.

True

C) To run a program that prints “hello world” to your screen your computer needs to use logic gates.

True

What does a CPU/processor do?

clicker.cs.illinois.edu

Q6

~Code~
340

a) It runs python, C, ~~and other code~~ ~~instructions.~~ F

b) It runs 1 assembly instruction at a time using registers and the ALU. machine code T

c) It runs many instructions in parallel by utilizing the ALU as register space. F

Explain to a neighbor...

What is the difference between a thread and a process? How are they handled differently and similarly by the Kernel?

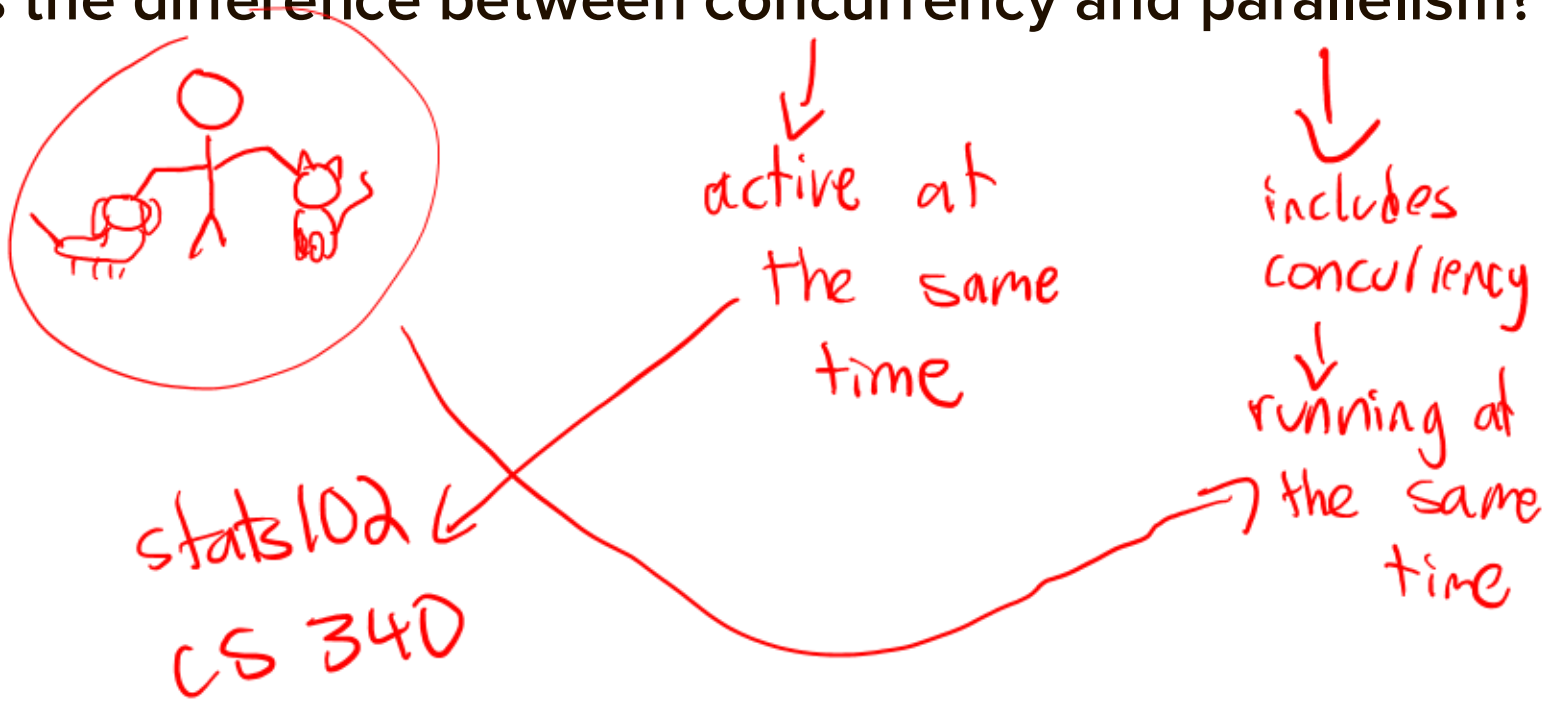
process – a running instance of a program
- each has its own VM

within a process are 1-many thread → program counter, register value

Kernel schedules based on threads in the CPU
- if the new thread is in a different process → load in new page table

Explain to a neighbor...

What is the difference between concurrency and parallelism?



Why Bits?

On hardware we can easily tell the difference between high and low voltage (1 or 0)



store information in 1s and 0s

bits



8 bits in 1 byte



2 Hex digits

Bits stored != interpreted value

0 0 1 0 0
16 8 4 2 1

- could represent a binary number
  - an address
  - a character - ascii

- could represent a negative number (2's complement)
- represent metadata in a utf-8 character
  - represent if something is used (MP allocator)
  - represent a set (MP3.2)

---

in C we can use
<< >>
AND OR XOR, Pointer math

Negative Numbers Review

4 bit

-7 —? stored

↘ 0111

flip 1000

add 1 1001

1010 stored

↘ 0101

0110

4 = 1 ↘ -6

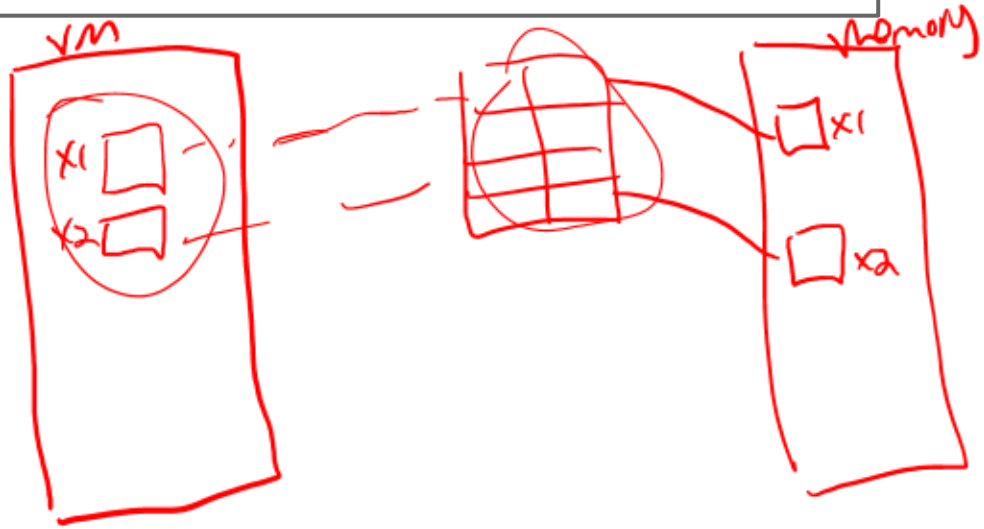
What prints?

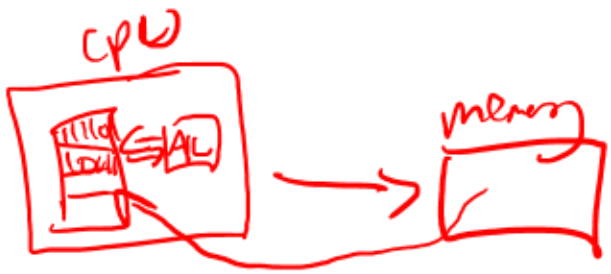
```
11  int main() {
12      unsigned char x1 = 0b11111001;
13      char x2 = 0b11111001;
14      printf("%u and %u", x1, x2);
15  }
```

clicker.cs.illinois.edu

Q7

~Code~
340





249 and -7

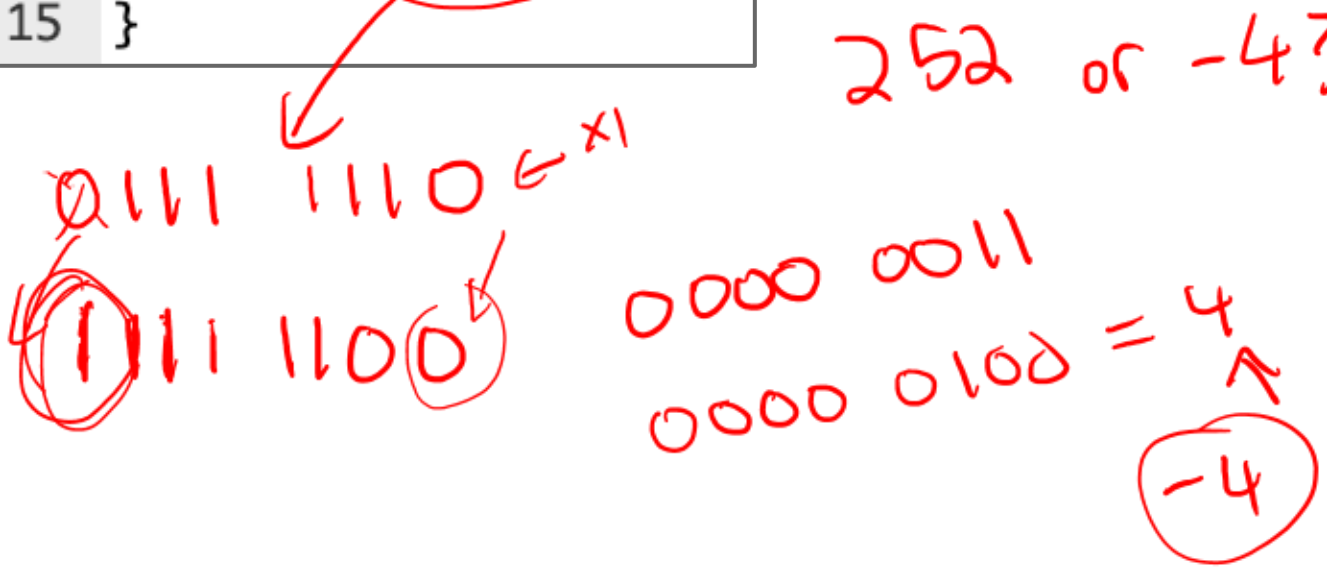
What prints?

```
11 int main() {
12   char x1 = 0x7E;
13   x1 = x1 << 1;
14   printf("%i\n", x1);
15 }
```

clicker.cs.illinois.edu

Q8

~Code~
340



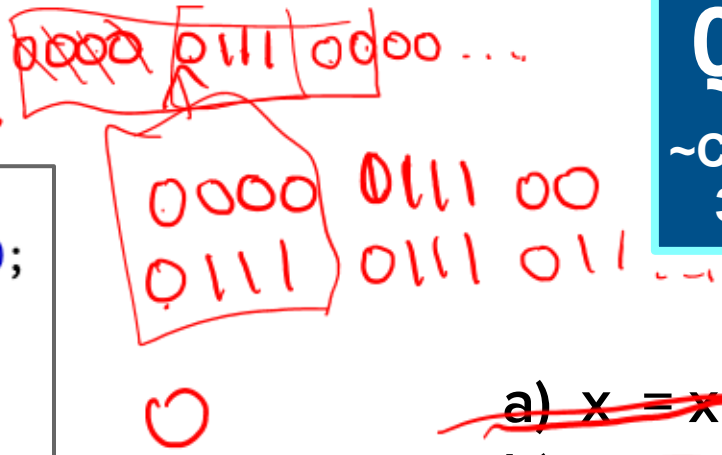
What code at line 14 would make the program print out a negative value?

clicker.cs.illinois.edu

Q9

~Code~
340

```
11 int main() {
12     int x = 0x07000000;
13
14     //??
15
16     printf("%i\n", x);
17 }
```



a) x = x << 4;

b) x = x << 10;

c) x = x ^ 0x77777777;

d) x = (~0) | x;



How many UTF-8 characters are encoded in these bytes?

clicker.cs.illinois.edu

Q10

~Code~
340

0x01 0xF7 0x88 0x84 0x98 0x1F

Recall that UTF-8 encodes code points in bytes using the following table:

```
0xxxxxxx
110xxxxx 10xxxxxx
1110xxxx 10xxxxxx 10xxxxxx
11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
```

Printing out the 9th bit

```
12  int main() {
13      void *ptr = malloc(500);
14
15      //how would I print out the 9th bit?
16
17
18
19
20
21
22      free(ptr);
23  }
```

Chat with a neighbor, what went wrong?

```c
int main() {
    void *ptr = malloc(500);

    //how would I print out the 9th bit?
    ptr = ptr + 1;
    char *ptr2 = (char*)ptr;
    unsigned char val = *ptr2;
    val = val >> 7;
    printf("%i\n", val);

    free(ptr);
}
```

```
main.c: In function ‘main’:
main.c:22:5: warning: ‘free’ called on pointer ‘ptr’ with nonzero offset
   22 |     free(ptr);
      |     ^~~~~~~~
main.c:13:17: note: returned from ‘malloc’
   13 |     void *ptr = malloc(500);
      |                 ^~~~~~~~~~
0
munmap_chunk(): invalid pointer
```

Chat with a neighbor, what is the definition of a race condition?

- racing to change or access some data
- Threads (more than 1)
- shared resources
- non deterministic behavior

This code sometimes prints 1 and sometimes 0, is there a race condition?

clicker.cs.illinois.edu

Q11

~Code~
340

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    void *ptr_front = malloc(500);

    //how would I print out the 9th bit?
    void* ptr = ptr + 1;
    char *ptr2 = (char*)ptr+1;
    unsigned char val = *ptr2;
    val = val >> 7;
    printf("%i\n", val);

    free(ptr_front);
}
```

no

```
4  char str[6] = {'h','e','l','l','o','\n'};
5
6  void *comp(void *mem){
7      char *replace = (char*)mem;
8      char *ptr = str;
9      while(*ptr != '\n'){
10         if(*ptr == replace[0]){
11             *ptr = replace[1];
12         }
13         ptr++;
14     }
15     return NULL;
16 }
17
18 int main() {
19     char replace0[2] = {'y', 'm'};
20     char replace1[2] = {'o', '0'};
21     char replace2[2] = {'h', 'y'};
22     pthread_t threads[3];
23     pthread_create(&threads[0], NULL, comp, replace0);
24     pthread_create(&threads[1], NULL, comp, replace1);
25     pthread_create(&threads[2], NULL, comp, replace2);
26     for(int i = 0; i < 3; i++){
27         pthread_join(threads[i], NULL);
28     }
29     printf("%s\n", str);
30 }
```

yellO
mellO
hello

clicker.cs.illinois.edu

Q12

~Code~
340

What are two possible outputs?

race condition

Synchronization Summary

Errors - race conditions
deadlock

Synchronization tools -

Mutex
reader/writer
condition variable

Exam 2 Format

- 5pts - Conceptual MCQs.
- 5pts - UTF-8 convert question
- 5pts - 1 making masks with bitwise operations variant (from hw)
- 15pts - Question about MP3.2 bitsets
- 15pts - Question about MP4 allocator

Practice Exam Question #1

MP3.2 asked you to edit mystery.c from an earlier MP to use bitsets instead of an array of char*'s. This helps speed up your code!

Explain how you used bitsets in MP3.2 and how using bitsets reduced the time it took for the code to run.

100% Answer

A bitset is a series of bytes that uses individual bits to indicate if an item is in the set. In this MP I used bit sets to represent which letters were present in each word. Each word was represented by an int (32 bits). Each letter in the word was indicated as in the word if it's corresponding bit was set in the int.

This improved the speed because I could use bit operations (specifically the & operation) to find the intersection between two words instead of looping through each letter. The & operation is faster because it uses fewer machine code instructions on the CPU than looping through each letter.

50% Answer

I used bitsets in MP3.2 to store information. A bitset improved the speed because I could use bit operations (specifically the & operation) to find the intersection between two words instead of looping through each letter. The & operation is faster because it uses fewer machine code instructions on the CPU than looping through each letter.

0% Answer

Using bitsets reduced the time it took for the program to run. I used bitsets to reduce the runtime.

Practice Exam Question #2

4. Shrink at end of heap

When deallocating (with myfree) or decreasing (with myrealloc) the size of the block at the end of the allocated memory, also return its memory to the unused memory set.

```
const char *test1(allocator *a) {
  void *tmp1 = a->malloc(4);
  void *tmp2 = a->malloc(4);

  a->free(tmp2);

  void *tmp3 = a->malloc(5);

  if(tmp3 > tmp2){
    return "SUCCESS";
  }
  else{
    return "FAIL";
  }
}
```

Explain why or why not this test would or would not verify the behavior from step 4.

Exam 2 - 100% Answer

This test does not verify step 4. Step 4 involves being able to reuse memory if it was deallocated at the end of the used memory. In the specific case above, tmp1 has allocated the first 4 bytes and then tmp2 allocates the next 4 bytes. Next, tmp2 frees its memory of 4 bytes at the end of the used memory. If step 4 is implemented correctly, this should make the address at tmp2 available to be allocated again. So when tmp3 requests 5 bytes, it should be allocated where tmp2 used to be. To check this we would have to see if tmp3 == tmp2. The current test checks if tmp3 > tmp2 which does not verify step 4.

Exam 2 - 50% Answer

This test does not verify step 4. To check this we would have to see if tmp3 == tmp2.

Exam 2 - 0% Answer

This test does verify step 4. In the specific case above, tmp1 has allocated the first 4 bytes and then tmp2 allocates the next 4 bytes. Next, tmp2 frees its memory of the 4 bytes but the algorithm so far does not keep a free list or reuse free'd memory. So when tmp3 requests 5 bytes, it should be allocated after the bytes originally needed for tmp2. To check this we would have to see if tmp3 > tmp2 which is what the test does.

Exam 2 - 0% Answer

This test does not verify step 4.